

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Tue 3 March 2015 - Part Theory

NAME:

MATRICOLA:

SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

1. Consider the following regular expression R , over the two-letter alphabet $\{a, b\}$:

$$R = (a^+ \mid ba \mid ab a)^* b$$

Answer the following questions:

- (a) Provide evidence that the regular expression R is ambiguous, by referring to the string $abab$.
 - (b) By using the Berry-Sethi method, construct a deterministic finite state automaton A that recognizes the language $L(R)$.
 - (c) Minimize the number of states of the automaton A obtained at the previous point (b) and find the equivalent minimal automaton A_{min} .
 - (d) (optional) Define a strictly right unilinear grammar G that generates the language $L(R)$. Grammar G should be as simple as possible in terms of the number of nonterminal symbols and rules. Also write a derivation of the string $abab$ according to grammar G .
-

2 Free Grammars and Pushdown Automata 20%

1. Consider a language L_1 of nested lists, possibly empty and of arbitrary depth. Every (sub)list is embraced in round brackets “(” and “)”, and may contain elements schematized by letter e . The whole string is one list embraced in round brackets.

Here are four sample nested lists s_i , with $i = 1, 2, 3, 4$:

$$s_1 = () \quad s_2 = (e e) \quad s_3 = ((e)) \quad s_4 = (e (e) e e)$$

Answer the following questions:

- (a) Write a non-extended (*BNF*) grammar G_1 , not ambiguous, that generates the language L_1 described above, and draw the syntax tree of the sample string s_4 .
- (b) Consider a modified list language L_2 , similar to L_1 but with hash separators “#”, which must be inserted between two consecutive letters e and nowhere else. For instance, the sample string s_4 above becomes as the string s_5 below:

$$s_5 = (e (e) e \# e)$$

Write a non-extended (*BNF*) grammar G_2 , not ambiguous, that generates the language L_2 described above, and draw the syntax tree of the sample string s_5 .

- (c) (optional) Consider another modified list language L_3 , which is similar to L_1 and contains all the strings of L_1 , but such that one closed square bracket “]” can be used at the string end to close all the preceding round brackets “(” that are still pending open. For instance, here is a sample string s_6 of such kind:

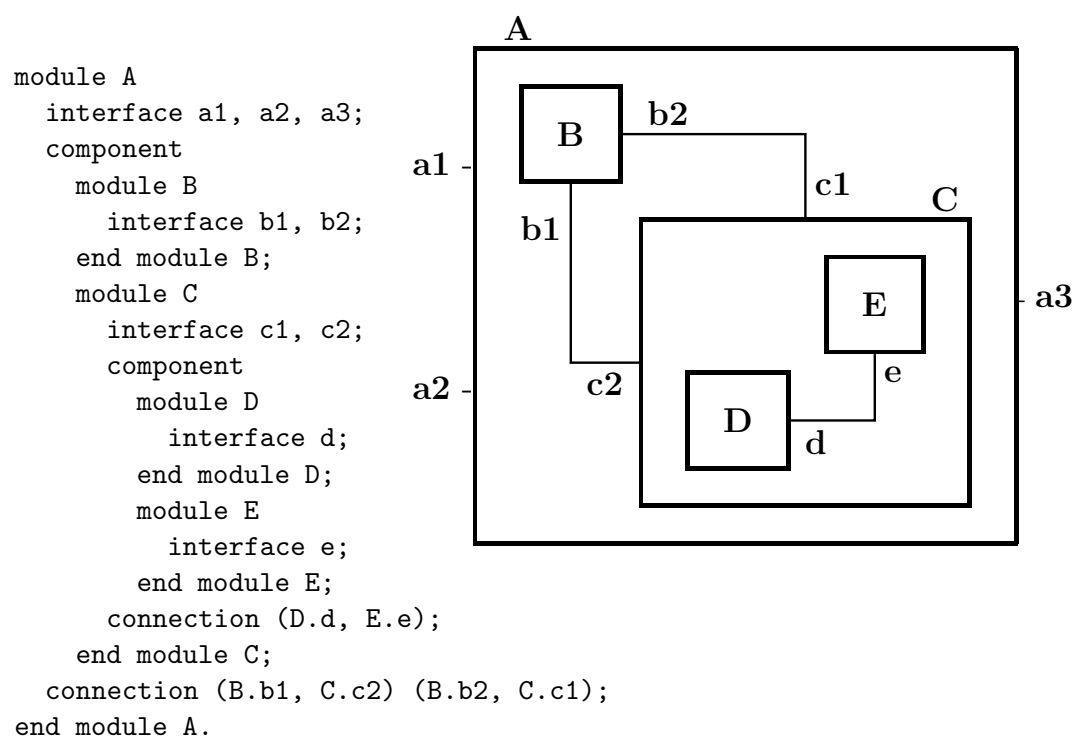
$$s_6 = (((() e) e (e]$$

where the 1st and 4th open round brackets are both closed by “]”. Write a non-extended (*BNF*) grammar G_3 , no matter if it is ambiguous, that generates the language L_3 described above, and draw the syntax tree of the sample string s_6 .

2. A textual language defines the structure of nested modules with connections. A program in such a language consists of the definition of one module, with these features:
- every module has an *interface* that consists of one or more items
 - a module may include *nested modules*, which in turn may include more modules, and so on at an arbitrary depth
 - a module may specify one or more *connections*: each connection links two interface items of immediately nested modules (one item per module)

The module and interface names are identifiers schematized by the terminal `id`.

For instance, the modular structure depicted in the figure below on the right is represented by the program reported on the left. Notice that the program includes only structural information related to the nesting relation between modules, to their interface items and to their connections, but that it ignores any other geometric/typographic information possibly conveyed by a graphic representation like that in the figure.

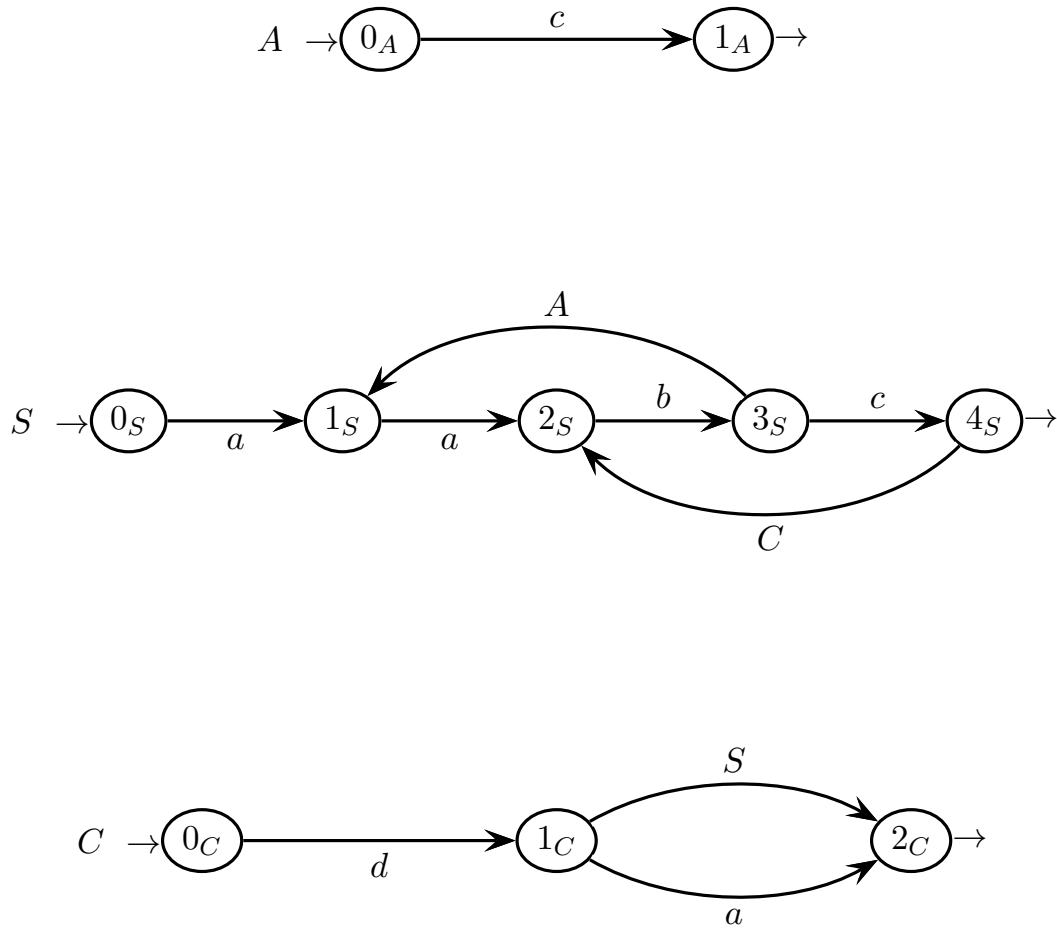


Answer the following questions:

- Write an extended grammar (*EBNF*) G_1 , not ambiguous, that reasonably models the language sketched above.
- (optional) Write an extended grammar (*EBNF*) G_2 , not ambiguous, for the following variation of the language:
 - a module may or may not have interface items
 - every module that immediately includes two or more modules with some interface items, necessarily includes also a *connection* section

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the following machine net (axiom S), over the four-letter alphabet $\{a, b, c, d\}$:

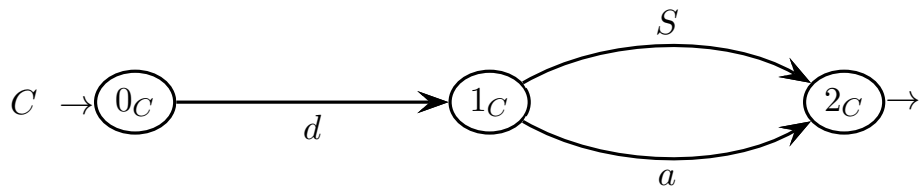
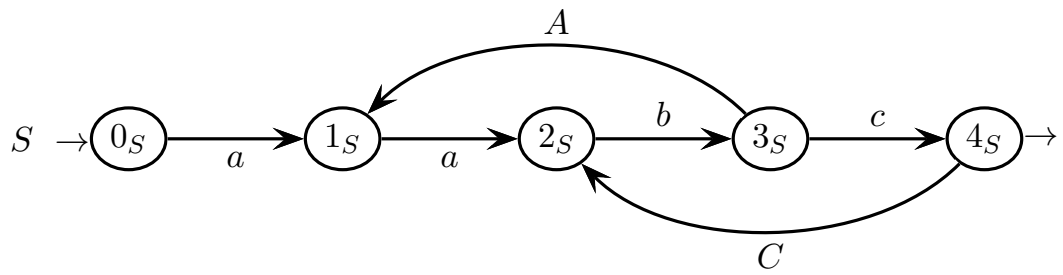
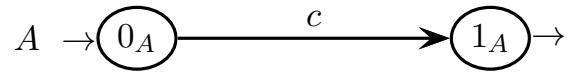


Answer the following questions:

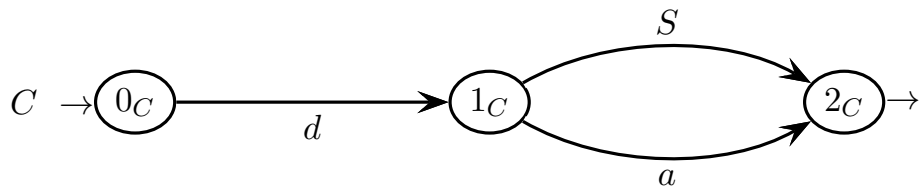
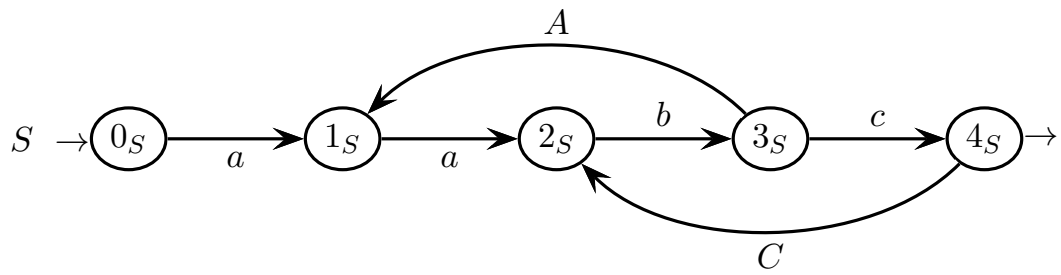
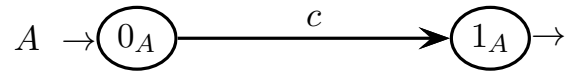
- (a) Draw the complete pilot graph of the machine net and say if the net is of type $ELR(1)$ or not (explain your answer). Is the $ELL(1)$ condition satisfied by the pilot or not (explain your answer)? Use the space left on the next pages.
- (b) Write the guide sets on each call and exit arrow of the machine net, and using these guide sets (as well as those on the terminal shift arcs) say if the net is of type $ELL(1)$ or not (explain your answer). Use the net on the next pages.
- (c) (optional) Examine the net and say if it is of type $ELL(k)$ for some $k \geq 2$ or not. This may require to find a few guide sets of a length greater than one, and to reason using them. Please answer concisely but rigorously. If you wish, you can use the net on the next pages.

question (a) - please draw here the **PILOT GRAPH** and write your explanation

question (b) - please compute here the **GUIDE SETS** and write your explanation



question (c) - please compute here the **GUIDE SETS** and write your explanation



4 Language Translation and Semantic Analysis 20%

1. Consider the following source grammar G_s (axiom S):

$$G_s \left\{ \begin{array}{l} S \rightarrow A X \mid A \\ X \rightarrow C S \mid C \\ A \rightarrow a A \mid a \\ C \rightarrow c C \mid c \end{array} \right.$$

which generates strings composed by alternated groups of letters a and c .

Answer the following questions:

- (a) Write a destination grammar G_d that defines a translation τ of the strings of the language $L(G_s)$, such that:
 - every group of letters a of **length two or more** is reduced to one letter a
 - and every group of letters c is enclosed within a pair of letters b and e

For instance:

$$a a c c c a c a \xrightarrow{\tau} a b c c c e a b c e a$$

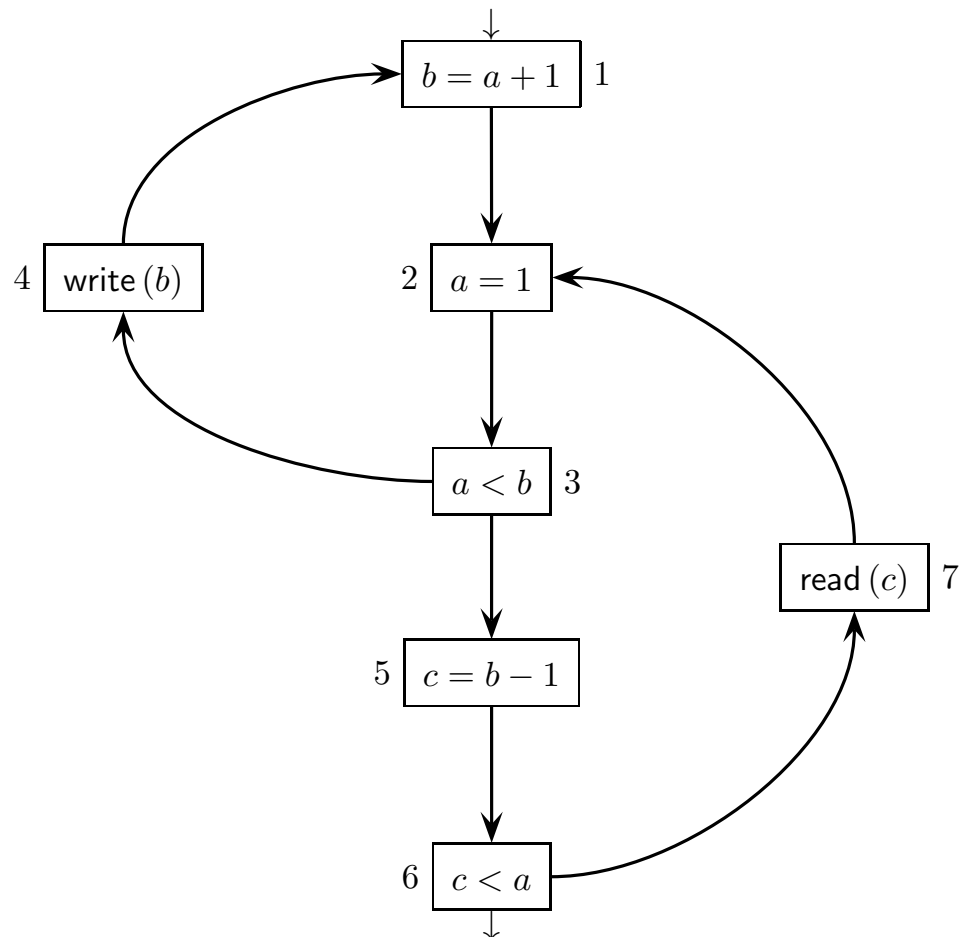
- (b) Modify the previous syntactic scheme, and if necessary also change the source grammar, in such a way that the source language is left unchanged and the translation τ is redefined as a new translation τ' , such that:
 - every group of letters a of length two or more is reduced to one letter a
 - and every group of letters c of **length two or more** is enclosed within a pair of letters b and e

For instance:

$$a a c c c a c a \xrightarrow{\tau'} a b c c c e a c a$$

- (c) (optional) Define a deterministic sequential (i.e., finite state) transducer T that computes the translation τ' defined at the previous point (b).

2. Consider the following control flow graph (*CFG*) of a program, with seven nodes (input node 1 and output node 6):



Answer the following questions:

- Find the live variables at all the nodes of the *CFG*, by using the flow equation method (for live variables). Use the tables on the next pages. Write the solution by the graph nodes (use the graph after the tables).
- (optional) Find the reaching definitions in the *CFG*, informally or again by using the flow equation method (for reaching definitions). Use the tables on the next pages. Write the solution by the graph nodes (use the graph after the tables).

<i>node</i>	<i>defined</i>
1	
2	
3	
4	
5	
6	
7	

<i>node</i>	<i>used</i>
1	
2	
3	
4	
5	
6	
7	

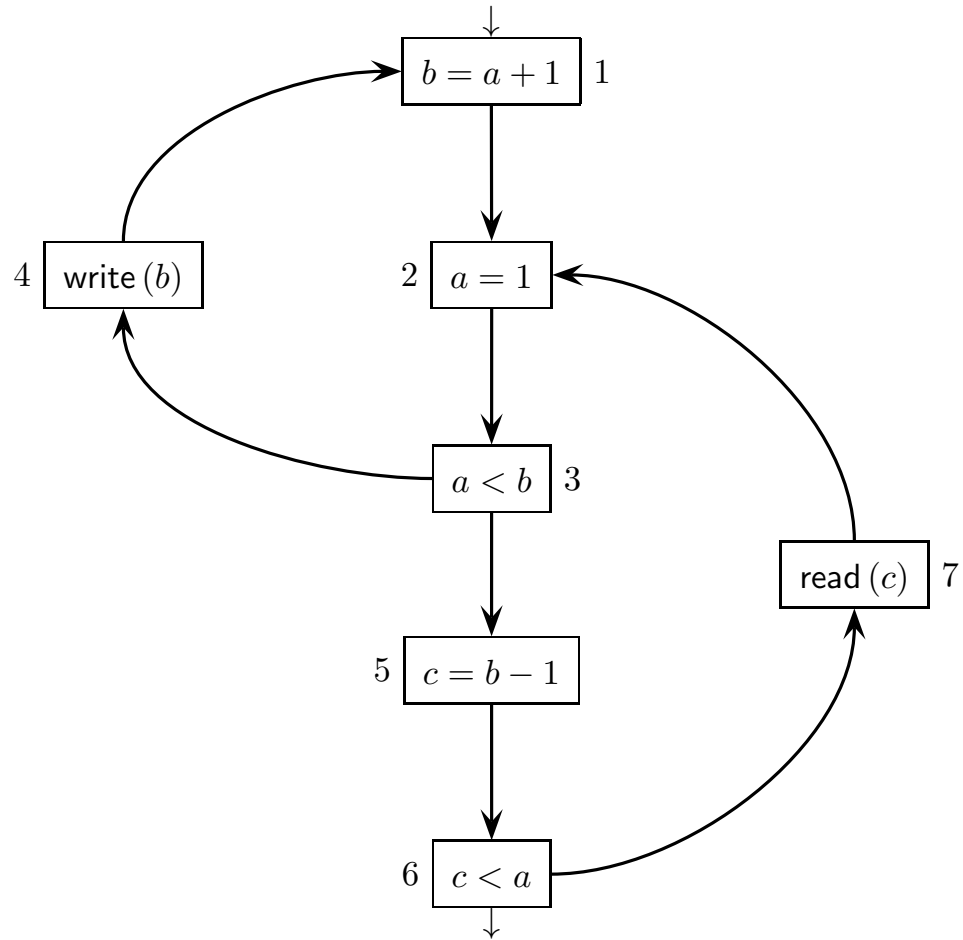
system of data-flow equations for **LIVE VARIABLES**

<i>node</i>	<i>in equations</i>	<i>out equations</i>
1		
2		
3		
4		
5		
6		
7		

iterative solution table of the system of data-flow equations (**LIVE VAR.S**)
(the number of columns is not significant)

	<i>initialization</i>		1		2		3		4		5		6	
#	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>
1														
2														
3														
4														
5														
6														
7														

please here write the **LIVE VARIABLES**



<i>node</i>	<i>defined</i>
1	
2	
3	
4	
5	
6	
7	

<i>node</i>	<i>suppressed</i>
1	
2	
3	
4	
5	
6	
7	

system of data-flow equations for **REACHING DEFINITIONS**

<i>node</i>	<i>in equations</i>	<i>out equations</i>
1		
2		
3		
4		
5		
6		
7		

iterative solution table of the system of data-flow equations (**REACHING DEF.S**)
(the number of columns is not significant)

	<i>initialization</i>		1		2		3		4		5		6	
#	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>
1														
2														
3														
4														
5														
6														
7														

please here write the **REACHING DEFINITIONS**

